

Конспект лекции 14: «Матрица». Двойная индексация и прокси-классы

Преподаватель: Сергей Шершаков

1. Проблема синтаксиса двойного индекса $m[i][j]$

Для интуитивной работы с матрицей разработчик хочет использовать привычный синтаксис двумерных массивов: $m[i][j]$. Однако в C++ оператор квадратных скобок `operator[]` принимает ровно один аргумент.

- Напрямую перегрузить `operator[][]` невозможно.
- Решением является паттерн проектирования Прокси (Proxy / Заместитель).

2. Реализация через вспомогательные классы Row и ConstRow

Идея заключается в разделении процесса индексации на два этапа:

1. Вызов $m[i]$ на объекте матрицы возвращает не число, а временный легкий промежуточный объект — строку (класс `Row`).
2. Второй вызов $[j]$ применяется уже к этому объекту `Row`, который, зная индекс строки и адрес матрицы, вычисляет и возвращает реальный элемент.

```
class Matrix {
public:
    // Вложенный прокси-класс для обычных матриц
    class Row {
    private:
        Matrix* _owner;
        size_t _rowInd;
    public:
        Row(Matrix* owner, size_t rowInd) : _owner(owner), _rowInd(rowInd) {}
        double& operator[](size_t colInd) {
            return _owner->_data[_rowInd * _owner->_colsNum + colInd];
        }
    };

    Row operator[](size_t ind) {
        return Row(this, ind);
    }
};
```

```
    }  
};
```

3. Константная безопасность (Const Correctness)

Если матрица передается в функцию по константной ссылке (*const Matrix& m*), обычный метод *operator[]* вызвать нельзя, так как он может позволить модифицировать данные.

- Необходимо продублировать логику для константного контекста.
- Вводится второй прокси-класс — *ConstRow*, у которого его собственный *operator[]* возвращает значение или константную ссылку (*const double&*), запрещая изменения.

```
class Matrix {  
public:  
    class ConstRow {  
    private:  
        const Matrix* _owner;  
        size_t _rowInd;  
    public:  
        ConstRow(const Matrix* owner, size_t rowInd) : _owner(owner), _rowInd(rowInd) {}  
        double operator[](size_t colInd) const {  
            return _owner->_data[_rowInd * _owner->_colsNum + colInd];  
        }  
    };  
  
    ConstRow operator[](size_t ind) const {  
        return ConstRow(this, ind);  
    }  
};
```